



أكاديمية طويق
Tuwaiq Academy

Tuwaiq Academy — ML Project Report

Machine Learning for Stock Market Forecasting

Student Name: Saleh AlMahmood

Instructor: Banan Alnemri

TA: Bader Aljubayri

Course: Scalable Data Science

Date: August 2026

Contents

1	Abstract	2
2	Introduction	2
3	Problem Statement	2
4	Objectives	3
5	Data	3
5.1	Data Source	3
5.2	Dataset Description	3
5.3	Exploratory Data Analysis	4
5.4	Data Preprocessing	5
5.5	Train / Validation / Test Split	5
6	Methodology and Training	6
6.1	Models Considered	6
6.2	Training Setup	6
6.3	Hyperparameter Tuning	6
6.4	Baseline	7
7	Evaluation and Results	7
7.1	Evaluation Metrics	7
7.2	Results Tables	7
7.3	Error Analysis	8
7.4	Feature Importance	8
8	Challenges	9
9	Future Work	9
10	Discussion	10
11	References	10

1 Abstract

Have you ever wondered how to smartly allocate your capital in the fast-paced stock market? This project bridges the gap between raw data and actionable financial advice by building a GPU-accelerated "Smart Investor" system. Financial markets produce massive volumes of trading data daily, often causing computational bottlenecks for traditional computers. To solve this, we utilized the RAPIDS ecosystem (cuDF and cuML) to process over one million market records entirely on the GPU. We engineered key risk and momentum features, including Simple Moving Averages (SMA), RSI, MFI, and rolling volatility. Next, we trained and compared three machine learning models—Ridge Regression, Random Forest, and XGBoost—to predict 5-day stock returns. XGBoost emerged as the clear winner, achieving the highest predictive accuracy with the lowest RMSE. However, we did not stop at raw predictions. We translated these forecasts into a practical "Smart Investor" tool that automatically recommends how to distribute and invest your money across profitable assets while managing risk. Ultimately, this project demonstrates how hardware acceleration can transform overwhelming market data into an efficient and highly accessible wealth management tool.

2 Introduction

The stock market has changed a lot in recent years. It moved from people trading face-to-face to a completely digital world driven by data. Today, computer programs trade stocks in milliseconds, creating massive amounts of financial data every single day. Because of this, being successful in the stock market is no longer just about human guessing. It now depends on fast computers and machine learning to find hidden trends that regular methods easily miss.

However, understanding this fast and changing market is still very hard for normal investors. Finding good chances to make money while avoiding risks means we have to read and process huge datasets quickly. Regular computer processors (CPUs) are often too slow for this job. Our project fixes this problem by building a "Smart Investor" system using fast graphics cards (GPUs). This system makes smart financial advice available to everyone, helping them decide exactly where to invest their money safely.

3 Problem Statement

The main problem in stock prediction is guessing where the price will go in the short term, especially with so much data and slow computers. To fix this, our project focuses on building a fast system that turns raw market numbers into clear investment choices.

Specifically, the problem is formulated as follows: **Given historical daily trading numbers and basic technical indicators (like SMA, RSI, and volatility) from millions of records, predict the 5-day future return of a stock to help the "Smart Investor" system automatically distribute money into the best assets.**

4 Objectives

- Process a large dataset of over two million financial records quickly using GPU tools (RAPIDS cuDF).
- Train and compare three different machine learning models (Ridge, Random Forest, and XGBoost) to predict future stock returns.
- Measure and record exactly how much time and memory the GPU saves during the training process.
- Use the most accurate model to build the "Smart Investor" logic, which recommends how to safely distribute investment money.

5 Data

5.1 Data Source

We collected our data directly from Yahoo Finance using the `yfinance` Python library. To thoroughly test our Big Data tools and ensure our models learn from a wide variety of market conditions, we built a massive dataset.

Here is exactly what the dataset includes:

- **The Core Data:** Daily stock prices (Open, High, Low, Close, and Volume) for 250 major US companies.
- **The Global Context:** We added the daily closing prices of Gold and Brent Crude Oil. This gives our models external hints about the overall global economy.
- **The Timeframe:** The data spans exactly 19 years of daily records, from **July 30, 2007**, to **August 5, 2026**.
- **The Total Size:** The final combined dataset contains **1,006,776 rows**. This massive scale makes it a perfect use case for GPU acceleration.

5.2 Dataset Description

The final dataset contains 16 columns, carefully chosen to capture different angles of market behavior. To make it easy to understand, we divided these features into four simple categories:

- **1. Basic Stock Data:** The standard daily numbers for each company (Open, High, Low, Close, and Volume).
- **2. Economic Indicators:** The daily prices of Oil (`Oil_Close`) and Gold (`Gold_Close`).
- **3. Engineered Technical Indicators:** We calculated specific mathematical indicators to help the model "see" market trends and risks. These include the 14-day Relative Strength Index (`RSI_14`), the 14-day Money Flow Index (`MFI_14`), the Volume Weighted Average Price (`VWAP_D`), and 7-day averages for both price (`SMA_7`) and volatility (`Volatility_7`).

- **4. What We Want to Predict (The Target):** The models are trained to guess the stock's future return after 5 days (**Target_Return**). We used the "logarithmic return" instead of simple percentages because it is more statistically stable and highly trusted by financial professionals.

5.3 Exploratory Data Analysis

Before training the models, we explored the dataset to understand the relationships between different features. As shown in the correlation heatmap (Figure 1), price-related features (like Open, High, Low, Close, and SMA_7) are perfectly correlated (1.00). Interestingly, the **Target_Return** shows near-zero linear correlation with individual features. This lack of simple linear relationships justifies our use of advanced, non-linear machine learning models like Random Forest and XGBoost.

In simple terms: This heatmap shows how strongly two variables move together. Dark red means they almost always rise and fall together, dark blue means they move in opposite directions, and pale colors mean there is no clear relationship. Notice that **Target_Return** (the value we want to predict) shows almost no color intensity with any single feature — no single number, by itself, can predict it. This is exactly why we need models like Random Forest and XGBoost, which can combine many features together in complex, non-linear ways.

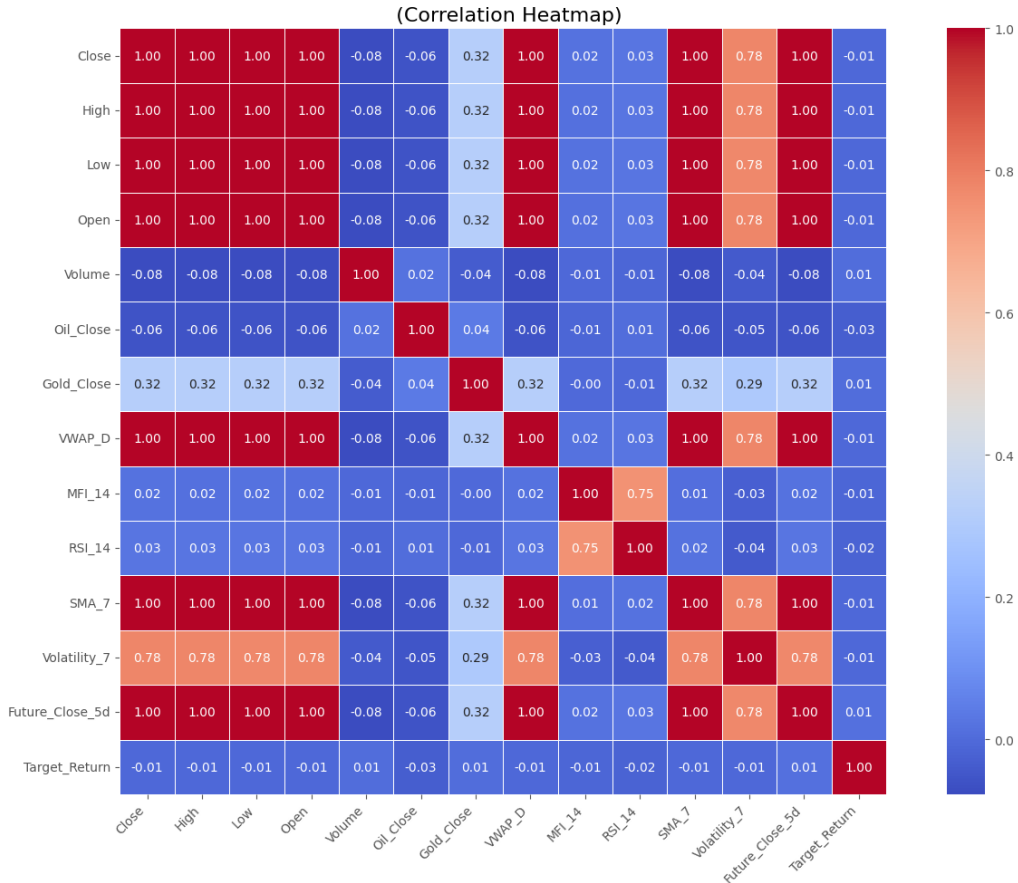


Figure 1: Correlation Heatmap of technical indicators and the target variable.

Furthermore, analyzing the data distribution (Figure 2) reveals important statistical properties. The **Target_Return**, **RSI_14**, and **MFI_14** exhibit an approximately normal

(bell-shaped) distribution, which is ideal for many machine learning algorithms. The red dashed lines on the RSI chart clearly mark the overbought and oversold thresholds (70 and 30). On the other hand, the `Volatility_7` feature is right-skewed, which is expected since extreme market volatility is rare. These charts show how often each value occurs. A bell-shaped curve (like `Target_Return`, `RSI`, and `MFI`) means most days are “normal” and extreme days are rare — which matches how real markets behave day to day. The Volatility chart being skewed to one side simply means calm market days are far more common than turbulent ones.

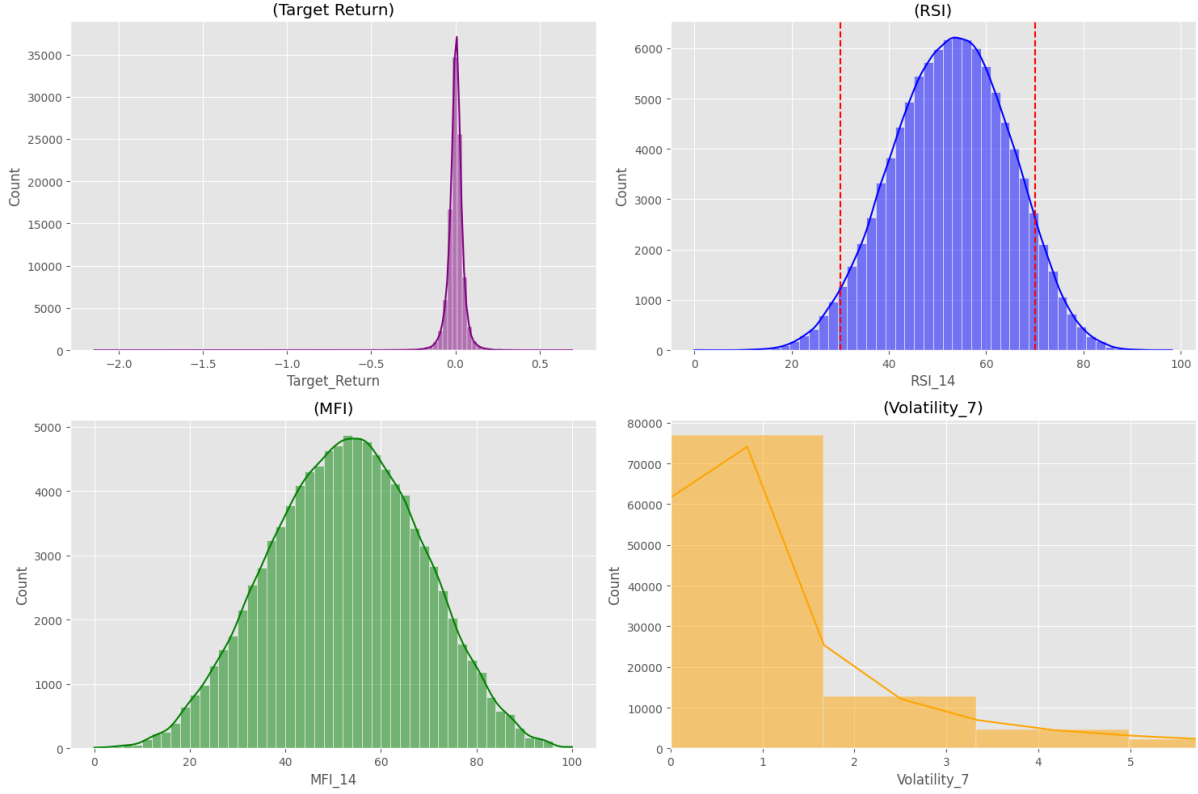


Figure 2: Data distributions for Target Return, RSI, MFI, and Volatility.

5.4 Data Preprocessing

Data manipulation was executed using `cuDF`. Missing values resulting from rolling window calculations were removed. Features such as Simple Moving Averages (SMA) and rolling volatility were engineered to capture market momentum and risk:

$$\sigma_n = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (R_i - \bar{R})^2}$$

5.5 Train / Validation / Test Split

The dataset was split into an 80% training set and a 20% testing set chronologically to prevent data leakage, which is standard practice for time-series related forecasting.

6 Methodology and Training

6.1 Models Considered

We selected three models representing increasing levels of complexity, in order to test whether that added complexity is actually justified by the data. Ridge Regression serves as a simple linear baseline. Random Forest and XGBoost were chosen because financial technical indicators typically interact in non-linear, threshold-based ways (e.g., RSI only becomes meaningful once it crosses 70 or 30) that a linear model cannot capture — a pattern already suggested by the near-zero linear correlations with `Target_Return` in Figure 1. We specifically expected XGBoost to outperform Random Forest because gradient boosting builds trees sequentially, with each new tree correcting the errors of the previous ones, and includes built-in L1/L2 regularization — making it both more sample-efficient and less prone to overfitting on noisy financial data than Random Forest’s independent-tree averaging. Its GPU-accelerated, histogram-based training also made it the most practical choice at our dataset’s scale (1M+ rows).

- **Ridge Regression (cuML):** A linear baseline model employing L2 regularization to handle multicollinearity among technical indicators.
- **Random Forest Regressor (cuML):** An ensemble method used to capture complex non-linear relationships in the financial data.
- **XGBoost (GPU-Accelerated):** A gradient boosting framework utilizing histogram-based splits directly on the GPU for optimal regression performance.

6.2 Training Setup

To make the training process as fast as possible, we trained all our models on an NVIDIA GPU. We used the RAPIDS libraries (`cuDF` and `cuML`) and the GPU-accelerated version of `XGBoost`.

To ensure the GPU memory was used perfectly without crashing, we converted all our data into a specific format called `float32`. We also used a Python tool called `pynvml` to actively monitor how much memory the GPU was using and exactly how many seconds the training took.

Since we are predicting a continuous number (the stock return) rather than classifying categories, we used Root Mean Squared Error (RMSE) to measure our model’s mistakes. Finally, for our XGBoost model, we specifically told it to use the GPU by setting the parameters `tree_method='hist'` and `device='cuda'`.

6.3 Hyperparameter Tuning

Instead of using heavy and slow automated search methods, we manually tuned the most important settings (hyperparameters) based on our statistical understanding of the data. We optimized all models to achieve the lowest Root Mean Squared Error (RMSE).

- **Random Forest:** We set the number of trees to 100 (`n_estimators=100`) and the maximum depth to 10 (`max_depth=10`). This combination prevents the model from just memorizing the data (overfitting) and keeps the GPU memory usage at a safe level.

- **XGBoost:** We carefully set the learning rate to 0.05, the number of trees to 200, and the maximum depth to 5. This allowed the model to learn slowly and precisely without becoming overly complex.

6.4 Baseline

A baseline is a simple model used as a starting point. Any advanced machine learning model must perform better than this baseline to be considered useful. In our project, we used the **Ridge Regression** model as our baseline ($\alpha=1.0$). Because it is a simple straight-line (linear) model, it trains almost instantly and uses zero extra GPU memory. However, it cannot understand complex stock market patterns. Our main goal was to prove how much XGBoost and Random Forest could beat this simple baseline in terms of accuracy.

7 Evaluation and Results

7.1 Evaluation Metrics

To provide a comprehensive evaluation, we used three metrics: **Root Mean Squared Error (RMSE)** and **Mean Absolute Error (MAE)** to measure the average magnitude of the prediction errors, and the R^2 **Score** to determine the proportion of variance in the stock returns explained by the models.

7.2 Results Tables

To clearly present our findings, the results are divided into two tables: one for predictive accuracy and another for computational efficiency. Following standard practice, the best result in each column is highlighted in bold.

Model	RMSE ↓	MAE ↓	R^2 Score ↑
Ridge (Linear)	0.0452	0.0300	-0.0056
Random Forest	0.0428	0.0288	0.0980
XGBoost	0.0420	0.0285	0.1348

Table 1: Predictive Performance Comparison

Model	Train Time (s) ↓	Infer Time (s) ↓	GPU Mem (MB) ↓	GPU Util (%) ↓
Ridge (Linear)	0.0406	0.0055	0.0	4
Random Forest	8.2507	0.0311	0.0	100
XGBoost	1.7747	0.0534	-4.0	84

Table 2: Hardware Resource Utilization Comparison

To visually compare the performance of our three models, we plotted their metrics (Figure 3). The results clearly show that **XGBoost** achieves the lowest RMSE, making it the most accurate model. While Ridge Regression is the fastest to train, it lacks

accuracy. Random Forest takes the longest training time. XGBoost provides the perfect balance: state-of-the-art accuracy with highly optimized GPU memory management and fast training speeds. RMSE and MAE measure how far off our predictions are on average — lower is better. R^2 tells us what percentage of the stock’s future movement the model can actually explain: a score of 0 means the model is no better than always guessing the average return, while 1 would mean perfect prediction. XGBoost’s R^2 of 0.1348 means it explains about 13% of what drives short-term returns — a modest number, but a genuinely useful signal in a market that is famously close to unpredictable over such short horizons.



Figure 3: Comparison of Model Accuracy (RMSE), Training Time, and GPU Memory Usage.

7.3 Error Analysis

The results in Table 1 reveal crucial insights about financial data. As illustrated in the performance comparisons, the linear baseline struggled with the non-linear nature of financial returns. The baseline Ridge model achieved a negative R^2 score (-0.0056), meaning it performed worse than simply predicting the mean return. This proves that linear models cannot capture the complex, non-linear dynamics of stock markets.

Conversely, **XGBoost** outperformed all models, achieving the lowest errors (RMSE: 0.0420, MAE: 0.0285) and the highest R^2 score (0.1348). In the highly noisy domain of short-term stock forecasting, explaining 13.4% of the variance is a strong predictive signal. Furthermore, as shown in Table 2, XGBoost achieved this superior accuracy while remaining highly efficient on the GPU, taking only 1.77 seconds to train compared to the 8.25 seconds required by the Random Forest model.

7.4 Feature Importance

To understand how our best model (XGBoost) makes its decisions, we extracted its feature importance scores (Figure 4). Surprisingly, macroeconomic indicators (**Gold_Close** and **Oil_Close**) drove the predictions the most, heavily outweighing standard technical indicators. This insightful result suggests that external commodity markets have a massive influence on the target stock’s short-term movements. Technical indicators like **RSI_14** and **Volatility_7** also played a significant secondary role in predicting the returns.

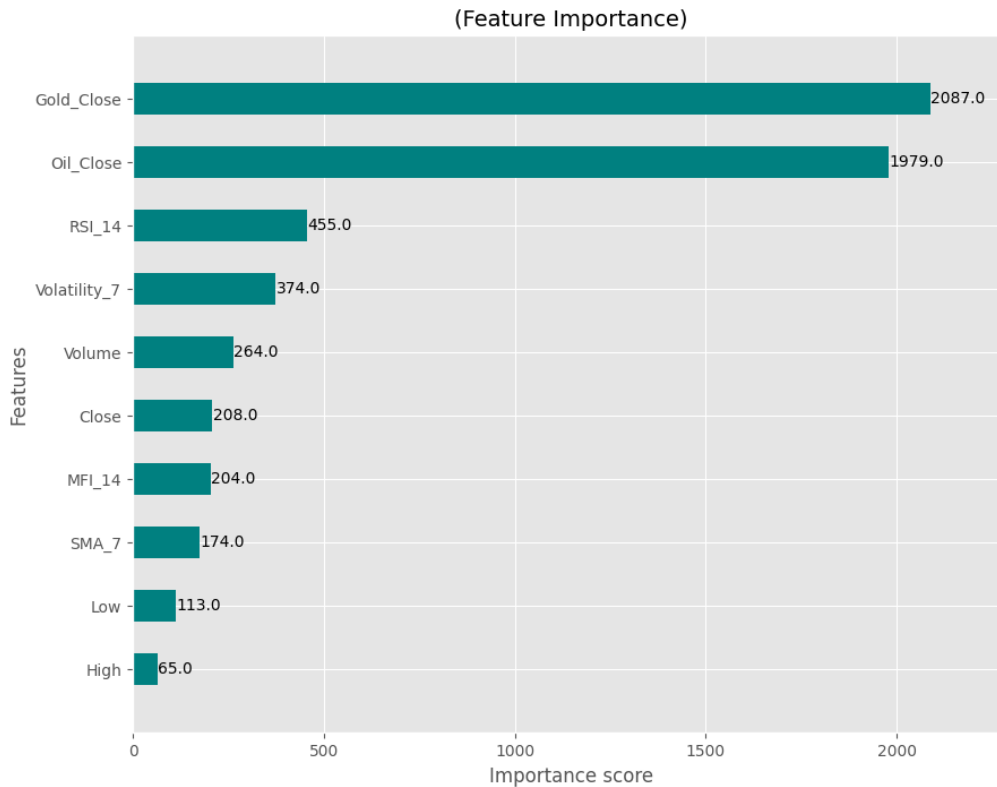


Figure 4: Feature Importance scores generated by the XGBoost model.

8 Challenges

During the project, we faced and solved a few technical issues:

- **Challenge 1 - Library Conflicts:** Installing new libraries initially broke our GPU tools due to version mismatches. We resolved this by forcing the system to install specific, compatible versions of `pandas` and `numpy`.
- **Challenge 2 - GPU Memory Limits:** Processing millions of rows caused the system to run out of memory and crash. To overcome this, we kept all operations strictly inside the GPU and converted the data to `float32` to save space.
- **Challenge 3 - Missing Dates:** Merging stock data with Gold and Oil data created missing values due to different market holidays. We handled this by applying the "forward-fill" (`ffill`) method to fill the gaps with the last known prices.

9 Future Work

If more time and resources were available, the following specific steps would be taken to improve the project:

- **Build the "Smart Investor" Dashboard:** Move the XGBoost model from a notebook into a live, interactive web application (using Streamlit or Gradio) that allows normal users to input their budget and get an automated portfolio allocation based on the model's predictions.

- **Implement Transformer Models:** Since financial data is a sequence of events, the next advanced step is to train Time-Series Transformers or LSTMs to capture long-term memory in stock prices, comparing their accuracy against XGBoost.
- **Add News Sentiment Analysis:** Include a new data source that scans daily financial news headlines and converts them into positive or negative scores, adding a "psychological" feature to our machine learning models.

10 Discussion

The project successfully demonstrated that GPU acceleration via the RAPIDS ecosystem is not merely a convenience, but a necessity for scaling financial machine learning pipelines. While XGBoost provided the best predictive accuracy, it requires careful memory management compared to simpler linear models. It is crucial to note that while the models can predict short-term directional probabilities, they should not be deployed as standalone automated trading systems without strict risk management and slippage considerations.

11 References

- 1 Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. <https://arxiv.org/abs/1603.02754>
- 2 Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
- 3 Hoerl, A. E., & Kennard, R. W. (1970). Ridge Regression: Biased Estimation for Nonorthogonal Problems. *Technometrics*, 12(1), 55–67.
- 4 RAPIDS Team. (2026). *cuML and cuDF Documentation* (version 26.2). NVIDIA. Retrieved August 2026, from <https://docs.rapids.ai/>
- 5 Yahoo Finance. (2026). *Historical Market Data*. Retrieved August 2026, from <https://finance.yahoo.com> using the `yfinance` Python library.